

NumPy II: orientation, broadcasting, and plotting what you compute

A flat array of three numbers is not a row and not a column — and almost everything confusing about NumPy follows from that one fact. Today we make orientation explicit, see broadcasting as NumPy supplying a missing orientation, use it deliberately to evaluate many models at once, and draw the results.

Unit 1 · Foundations

80 minutes

Lab 4 · Vectorise and Plot

Laptops open

Where we are

On Friday an array became a thing with a shape, and `.shape` became the first thing you print. Today has one central idea and two consequences:

1. **The idea:** an array's shape records not only *how many* numbers it holds but *how they are laid out* — and the 1-D shape `(n,)` has no layout at all.
2. **Consequence one:** broadcasting, the rule that lets arrays of different shapes combine.
3. **Consequence two:** a particular wrong answer that arrives without any error message.

Then plotting — and we will use the plot to see broadcasting do useful work, not just to draw pretty curves.

Three arrays that hold the same numbers

Same three values, three different shapes, three different behaviours.

1

type these and print each shape

PYTHON

```
import numpy as np

a = np.array([1, 2, 3])           # shape (3,)    - one axis
c = np.array([[1], [2], [3]])    # shape (3, 1) - 3 rows, 1 column
r = np.array([[1, 2, 3]])        # shape (1, 3) - 1 row, 3 columns

print(a.shape, c.shape, r.shape)
(3,) (3, 1) (1, 3)
```

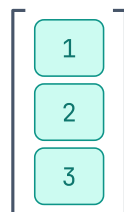
SAME NUMBERS, THREE LAYOUTS

(3,)
one axis — a plain sequence



not a row, not a column
a.T is still (3,) — nothing to flip

(3, 1)
two axes — a column



3 rows × 1 column

(1, 3)
two axes — a row



1 row × 3 columns

The number of entries in the shape tuple is the number of **axes**. `c` and `r` have two axes, so it makes sense to call one a column and the other a row. `a` has only one axis. Asking whether `a` is a row or a column is like asking whether a shopping list is horizontal: the list does not say. That is why transposing it does nothing:

.T swaps axes — and a 1-D array has only one

PYTHON

```
print(a.T.shape)    (3,)    ← unchanged
print(c.T.shape)    (1, 3)  ← column became a row
```

The three tools for giving an array an orientation

You will need these in Part 3, so we meet them now, while nothing is going wrong.

Expression	From (3,) you get	Read it as
<code>a.reshape(-1, 1)</code>	(3, 1)	"make it a column; work out the row count yourself" — the <code>-1</code> means <i>whatever fits</i> .
<code>a[:, None]</code>	(3, 1)	The same thing, shorter: keep every element (<code>:</code>), and insert a new axis of size 1 after it (<code>None</code>).
<code>a[None, :]</code>	(1, 3)	New axis of size 1 <i>first</i> : a row.
<code>c.ravel()</code>	(3,)	The reverse: flatten any shape back to one axis, discarding orientation.



Predict, then check — two minutes

`np.zeros(5)[:, None].shape`

`np.zeros((5,1)).ravel().shape`

`np.zeros((2,3)).T.shape`

`np.zeros(6).reshape(-1, 2).shape`

Answers: `(5, 1)`, `(5,)`, `(3, 2)`, `(3, 2)`.

Broadcasting: NumPy fills in what is missing

2

Two arrays of different shapes can combine element by element, provided NumPy can make their shapes agree by repeating size-1 axes.

You have already used it. In `a + 10`, the single number 10 behaves as if it were copied into every position of `a`. Broadcasting is the general version of that: an axis of size 1 behaves as if it were copied as many times as needed.

THE RULE — TWO STEPS, APPLIED TO THE SHAPE TUPLES

Step 1 · Give both arrays the same number of axes. If one has fewer, add axes of size 1 *at the front*. So `(4,)` is treated as `(1, 4)` — a row.

Step 2 · Compare axis by axis. At each axis the two sizes must be equal, or one must be 1. A size-1 axis is repeated to match the other. If neither holds at any axis, NumPy raises an error.

Why the missing axes go at the front

Short answer: in NumPy the **last axis is the innermost one**, and a 1-D array has exactly the structure of one innermost run. The memory layout makes this concrete. Run this first:

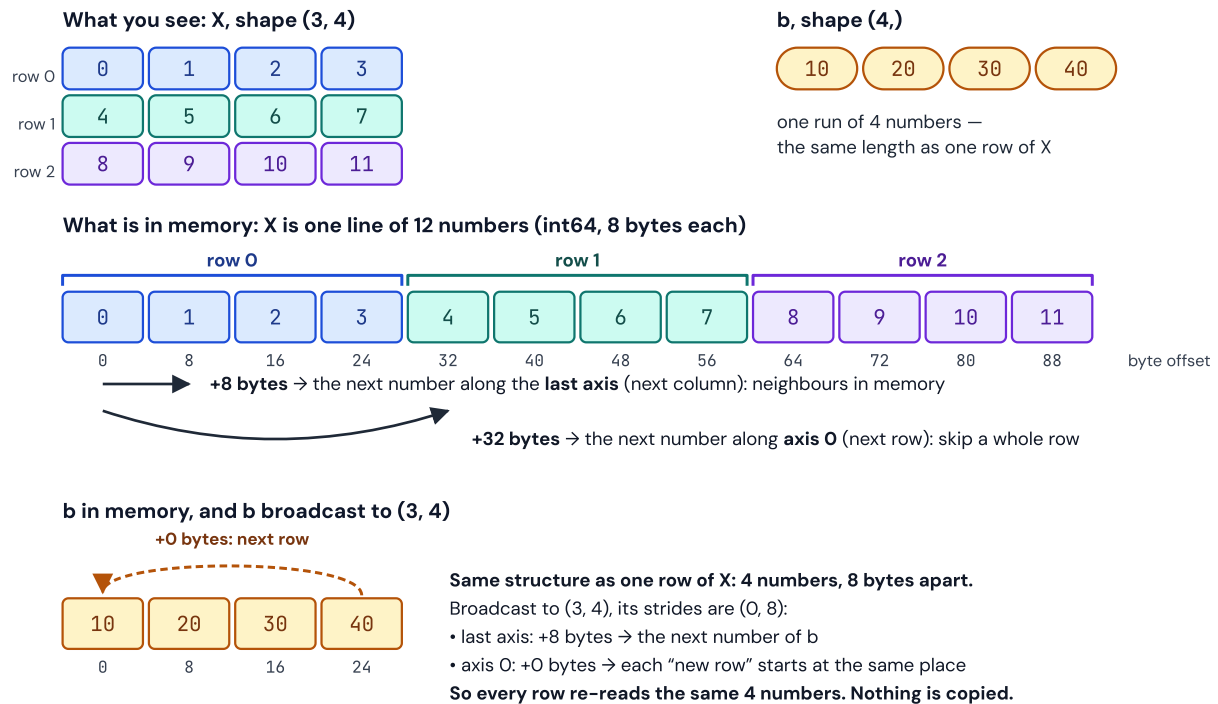
Look at how `X` and `b` are actually stored

PYTHON

```
X = np.arange(12).reshape(3, 4)
b = np.array([10, 20, 30, 40])

print(X.ravel())      # the order the 12 numbers sit in memory
[ 0  1  2  3  4  5  6  7  8  9 10 11]
print(X.strides)      # bytes to step one place along (axis 0, axis 1)
(32, 8)
print(b.strides)      # bytes to step one place along b's only axis
(8,)
```

THE GRID YOU SEE, THE LINE IN MEMORY, AND WHAT BROADCASTING DOES WITH B



Read the picture from top to bottom.

- 1. Memory is one line.** The grid is how we draw X; in memory it is 12 numbers in a row, stored row after row. The *strides* (32, 8) say how far to jump: 8 bytes to the next column, 32 bytes (four numbers) to the next row.
- 2. The last axis is innermost.** Moving along the last axis visits neighbours; moving along axis 0 skips whole rows. So X is "3 runs of 4", and the 4 belongs to the inside.
- 3. b is one run of 4.** Its stride is also 8. Structurally it is the same thing as one row of X. Matching b's only axis with X's *last* axis pairs b with each row, number by number — which is what `X + b` should mean.
- 4. Adding an axis at the front** is how NumPy writes that down: (4,) becomes (1, 4), "one row of 4". Step 2 then repeats that one row 3 times. If NumPy matched from the left instead, b's 4 would meet X's 3 rows, which neither makes sense nor fits.

You can watch the repeat happen without any copying:

the "repeated" row is really a stride of 0

PYTHON

```
B = np.broadcast_to(b, (3, 4))
print(B)
[[10 20 30 40]
 [10 20 30 40]
 [10 20 30 40]]
print(B.strides)    # moving to the "next row" moves 0 bytes
(0, 8)
```

A stride of 0 on axis 0 means "to get to the next row, do not move": every row reads the same four numbers. That is exactly the grey "behaves-as-if" entries in the worked cases below.



A precise footnote

The broadcasting rule is defined on *shapes*, not on memory. NumPy can also store X column by column — `F = np.asfortranarray(X)` has strides `(8, 24)` — and `F + b` gives exactly the same result as `X + b`. The memory picture explains why the convention is the natural one: in NumPy's default layout, and in the way you type nested lists like `[[0, 1, 2, 3], [4, 5, 6, 7], ...]`, the last axis is the innermost, and a flat list looks like one inner list. But the rule itself is a convention about shapes.

In practice the whole section comes down to one sentence: **a 1-D array always behaves as a row**. Hold on to it; Part 3 depends on it.

Case A • one value per column: $(3, 4) + (4,)$

Step 1 turns `b` from `(4,)` into `(1, 4)`. Step 2: axis 0 compares 3 with 1, so the single row is repeated three times; axis 1 compares 4 with 4, fine. Result `(3, 4)`. Here are the numbers — grey entries are the repeats NumPy behaves as if it had made:

$$\begin{array}{c} \begin{pmatrix} 0 & 1 & 2 & 3 \\ 4 & 5 & 6 & 7 \\ 8 & 9 & 10 & 11 \end{pmatrix} + \begin{pmatrix} 10 & 20 & 30 & 40 \\ 10 & 20 & 30 & 40 \\ 10 & 20 & 30 & 40 \end{pmatrix} = \begin{pmatrix} 10 & 21 & 32 & 43 \\ 14 & 25 & 36 & 47 \\ 18 & 29 & 40 & 51 \end{pmatrix} \\ X (3, 4) \qquad \qquad b (4,) \rightarrow (1, 4) \rightarrow \text{repeated down} \qquad \qquad (3, 4) \end{array}$$

This is exactly the `+ b` in `X @ W + b` to be introduced when we discuss Artificial Neural Networks later.

Case B • one value per row: (3, 4) + (3, 1)

$$\begin{array}{|c|c|c|c|} \hline 0 & 1 & 2 & 3 \\ \hline 4 & 5 & 6 & 7 \\ \hline 8 & 9 & 10 & 11 \\ \hline \end{array} + \begin{array}{|c|c|c|c|} \hline 100 & 100 & 100 & 100 \\ \hline 200 & 200 & 200 & 200 \\ \hline 300 & 300 & 300 & 300 \\ \hline \end{array} = \begin{array}{|c|c|c|c|} \hline 100 & 101 & 102 & 103 \\ \hline 204 & 205 & 206 & 207 \\ \hline 308 & 309 & 310 & 311 \\ \hline \end{array}$$

$\times (3, 4)$ $c (3, 1) \rightarrow$ repeated across $(3, 4)$

Case C • both sides stretch: (3, 1) + (1, 3)

Now each array has a 1 where the other does not. Both are repeated, and the result is a full table: every column entry combined with every row entry.

$$\begin{array}{|c|c|c|} \hline 10 & 10 & 10 \\ \hline 20 & 20 & 20 \\ \hline 30 & 30 & 30 \\ \hline \end{array} + \begin{array}{|c|c|c|} \hline 1 & 2 & 3 \\ \hline 1 & 2 & 3 \\ \hline 1 & 2 & 3 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 11 & 12 & 13 \\ \hline 21 & 22 & 23 \\ \hline 31 & 32 & 33 \\ \hline \end{array}$$

$(3, 1) \rightarrow$ across $(1, 3) \rightarrow$ down $(3, 3)$ — an addition table

This "every-with-every" table is genuinely useful — Part 5 uses it to test four candidate models against thirty data points in one line. It is also what produces the silent bug in Part 3. Same mechanism; the only difference is whether you meant it.



"Repeated" means behaves-as-if, not copied

NumPy never builds the grey entries in memory. It reads the same stored value again each time it needs one. That is why broadcasting a bias across a million rows costs nothing extra in memory — and why it is fast.

The whole rule, one example per case

Each card below is one case of the rule, with real numbers. Read the header first (the two shapes, the result, and the shapes after step 1), then check it against the arrays. Grey entries are the repeats NumPy behaves as if it had made. Flat (1-D) arrays are drawn as rounded chips with no brackets, to show that they have no orientation until step 1 makes them a row.

$$(3, 4) + (4,) \rightarrow (3, 4)$$

after step 1: (3, 4) vs (1, 4)

$$\begin{pmatrix} 0 & 1 & 2 & 3 \\ 4 & 5 & 6 & 7 \\ 8 & 9 & 10 & 11 \end{pmatrix} + \begin{pmatrix} 10 & 20 & 30 & 40 \end{pmatrix} \Rightarrow \begin{pmatrix} 10 & 20 & 30 & 40 \\ 10 & 20 & 30 & 40 \\ 10 & 20 & 30 & 40 \end{pmatrix} = \begin{pmatrix} 10 & 21 & 32 & 43 \\ 14 & 25 & 36 & 47 \\ 18 & 29 & 40 & 51 \end{pmatrix}$$

$\times (3, 4)$ $b (4,)$ treated as (1, 4), repeated down $(3, 4)$

Axis 0: 3 vs 1 $\rightarrow$ the single row repeats 3 times. Axis 1: 4 vs 4 $\rightarrow$ match. One value per *column* (Case A).

$$(3, 4) + (3, 1) \rightarrow (3, 4)$$

after step 1: (3, 4) vs (3, 1) – nothing to add

$$\begin{pmatrix} 0 & 1 & 2 & 3 \\ 4 & 5 & 6 & 7 \\ 8 & 9 & 10 & 11 \end{pmatrix} + \begin{pmatrix} 100 & 100 & 100 & 100 \\ 200 & 200 & 200 & 200 \\ 300 & 300 & 300 & 300 \end{pmatrix} = \begin{pmatrix} 100 & 101 & 102 & 103 \\ 204 & 205 & 206 & 207 \\ 308 & 309 & 310 & 311 \end{pmatrix}$$

$\times (3, 4)$ $c (3, 1)$, repeated across $(3, 4)$

Axis 0: 3 vs 3 $\rightarrow$ match. Axis 1: 4 vs 1 $\rightarrow$ the single column repeats 4 times. One value per *row* (Case B).

$$(3, 1) + (1, 3) \rightarrow (3, 3)$$

after step 1: (3, 1) vs (1, 3)

$$\begin{pmatrix} 10 & 10 & 10 \\ 20 & 20 & 20 \\ 30 & 30 & 30 \end{pmatrix} + \begin{pmatrix} 1 & 2 & 3 \\ 1 & 2 & 3 \\ 1 & 2 & 3 \end{pmatrix} = \begin{pmatrix} 11 & 12 & 13 \\ 21 & 22 & 23 \\ 31 & 32 & 33 \end{pmatrix}$$

$(3, 1)$, repeated across $(1, 3)$, repeated down $(3, 3)$

Each side has a 1 where the other does not, so *both* repeat. Every row value meets every column value: an addition table (Case C).

$(3, 1) + (3,) \rightarrow (3, 3)$

after step 1: $(3, 1)$ vs $(1, 3)$

$$\begin{array}{c} \begin{pmatrix} 10 \\ 20 \\ 30 \end{pmatrix} \\ (3, 1) - \text{a column} \end{array} + \begin{array}{c} \textcircled{1} \textcircled{2} \textcircled{3} \\ (3,) - \text{flat} \end{array} \Rightarrow \begin{array}{c} \begin{pmatrix} 10 & 10 & 10 \\ 20 & 20 & 20 \\ 30 & 30 & 30 \end{pmatrix} \\ \text{column, repeated across} \end{array} + \begin{array}{c} \begin{pmatrix} 1 & 2 & 3 \\ 1 & 2 & 3 \\ 1 & 2 & 3 \end{pmatrix} \\ \text{flat} \rightarrow \text{row } (1, 3), \text{ repeated} \\ \text{down} \end{array} = \begin{array}{c} \begin{pmatrix} 11 & 12 & 13 \\ 21 & 22 & 23 \\ 31 & 32 & 33 \end{pmatrix} \\ (3, 3) \end{array}$$

The same as the previous card. The flat array silently became a row, so a column plus a flat array of the same length gives a 3×3 table, not three sums. This is the silent bug of Part 3.

$(2, 3, 2) + (3, 1) \rightarrow (2, 3, 2)$

after step 1: $(2, 3, 2)$ vs $(1, 3, 1)$

$$\begin{array}{c} \begin{array}{cc} [0] & [1] \\ \begin{pmatrix} 0 & 1 \\ 2 & 3 \\ 4 & 5 \end{pmatrix} & \begin{pmatrix} 6 & 7 \\ 8 & 9 \\ 10 & 11 \end{pmatrix} \end{array} \\ A (2, 3, 2): \text{two } 3 \times 2 \text{ blocks} \end{array} + \begin{array}{c} \begin{pmatrix} 100 \\ 200 \\ 300 \end{pmatrix} \\ c (3, 1) \end{array} \Rightarrow \begin{array}{c} \begin{array}{cc} [0] & [1] \\ \begin{pmatrix} 100 & 100 \\ 200 & 200 \\ 300 & 300 \end{pmatrix} & \begin{pmatrix} 100 & 100 \\ 200 & 200 \\ 300 & 300 \end{pmatrix} \end{array} \\ (1, 3, 1), \text{ repeated across and} \\ \text{into both blocks} \end{array} = \begin{array}{c} \begin{array}{cc} [0] & [1] \\ \begin{pmatrix} 100 & 101 \\ 202 & 203 \\ 304 & 305 \end{pmatrix} & \begin{pmatrix} 106 & 107 \\ 208 & 209 \\ 310 & 311 \end{pmatrix} \end{array} \\ (2, 3, 2) \end{array}$$

Step 1 adds one axis at the front: $(3, 1) \rightarrow (1, 3, 1)$. Axis 0: 2 vs 1 $\rightarrow$ repeat for both blocks. Axis 1: 3 vs 3 $\rightarrow$ match. Axis 2: 2 vs 1 $\rightarrow$ repeat across. Same rule, one more axis.

$(3, 4) + (3,) \rightarrow$ **error**

after step 1: $(3, 4)$ vs $(1, 3)$

$$\begin{array}{c} \begin{pmatrix} 0 & 1 & 2 & 3 \\ 4 & 5 & 6 & 7 \\ 8 & 9 & 10 & 11 \end{pmatrix} \\ X (3, 4): \text{rows of length 4} \end{array} + \begin{array}{c} \textcircled{1} \textcircled{2} \textcircled{3} \\ d (3,) \rightarrow \text{row } (1, 3): \text{length 3} \end{array} \quad \times$$

Axis 0: 3 vs 1 is fine. Axis 1: **4 vs 3** — neither is 1. A row of 3 numbers cannot line up with rows of 4. If you meant one value per row, write `d[:, None]` to make it $(3, 1)$: that is the second card.

$(3,) + (4,) \rightarrow$ **error**

after step 1: $(3,)$ vs $(4,)$ – nothing to add

$$\begin{array}{ccccccc} \textcircled{1} & \textcircled{2} & \textcircled{3} & + & \textcircled{10} & \textcircled{20} & \textcircled{30} & \textcircled{40} & \times \\ (3,) & & & & (4,) & & & & \end{array}$$

One axis each: 3 vs 4, neither is 1. Three things cannot be paired with four. No reshape makes this meaningful – check the data instead.

Summary	A	B	Result
One value per column	$(3, 4)$	$(4,)$	$(3, 4)$
One value per row	$(3, 4)$	$(3, 1)$	$(3, 4)$
Column with row: every-with-every	$(3, 1)$	$(1, 3)$	$(3, 3)$
Column with flat: also every-with-every	$(3, 1)$	$(3,)$	$(3, 3)$
More axes, same rule	$(2, 3, 2)$	$(3, 1)$	$(2, 3, 2)$
Last axis disagrees	$(3, 4)$	$(3,)$	error
Different lengths	$(3,)$	$(4,)$	error

Reading the error, and fixing it correctly

X is $(3, 4)$. You meant to add one value per row.

ERROR

```
X = np.arange(12).reshape(3, 4)
d = np.array([1, 2, 3])
X + d
```

ValueError: operands could not be broadcast together with shapes (3,4) (3,)

The message gives both shapes. Diagnose with the rule: `d` became a row `(1, 3)`, and its 3 met X's 4. But you wanted one value *per row* – a column. So give `d` the orientation you meant:

state the intended orientation

FIXED

```
X + d[:, None]          # d is now (3, 1): Case B
[[ 1  2  3  4]
 [ 6  7  8  9]
 [11 12 13 14]]
```



Not every error should be "fixed" with a reshape

$(3,) + (4,)$ means you are combining a list of three things with a list of four. No orientation makes that meaningful. If you reshape one to $(3, 1)$, the error disappears and you get a $(3, 4)$ every-with-every table — almost certainly not what you wanted. Reshape only when you can say in words which orientation each array is *supposed* to have. Otherwise the error is telling you the data itself is wrong.

Checkpoint — predict the shape, or "error"

$(5, 2) + (2,)$

$(5, 2) + (5,)$

$(5, 1) + (2,)$

$(4, 1, 3) + (2, 1)$

ANSWER

$(5, 2)$ — the $(2,)$ becomes a row and repeats down.

error — $(5,)$ becomes $(1, 5)$, and 5 meets 2. If you meant one value per row, write $[:, \text{None}]$.

$(5, 2)$ — Case C: the column repeats across, the row repeats down.

$(4, 2, 3)$ — $(2, 1)$ becomes $(1, 2, 1)$; axis by axis: $4/1 \rightarrow 4$, $1/2 \rightarrow 2$, $3/1 \rightarrow 3$.

When broadcasting succeeds and the answer is wrong

3

A loud error is easy to fix. This case produces a perfectly reasonable-looking number.

You have three predictions and three true values and you want the mean squared error. The predictions are a 1-D array, which is what most NumPy calculations produce. The targets were stored as a column, `(3, 1)` — which is what you get whenever data is kept as "one row per example, one column per quantity" and you slice out a single column with `data[:, [2]]` or a reshape. You will meet both shapes in session 10.

predict the output before running it

PYTHON

```
p = np.array([1., 2., 3.])          # predictions   shape (3,)
t = np.array([[1.], [2.], [3.]])    # targets       shape (3, 1)

mse = ((p - t) ** 2).mean()
print(mse)
```

Every prediction equals its target. What is printed?

Use the cards from Part 2. Which card does `(3,) - (3, 1)` match?

ANSWER

It prints **1.3333333333333333**, not 0.

Why. By the rule, the 1-D `p` behaves as a row `(1, 3)` and `t` is a column `(3, 1)`. That is Case C: the result is a `(3, 3)` table of every prediction minus every target.

$$\begin{pmatrix} 0 & 1 & 2 \\ -1 & 0 & 1 \\ -2 & -1 & 0 \end{pmatrix}$$

`p - t`, shape (3, 3)

The diagonal (green) holds the three comparisons you wanted — all zero. The six red entries compare a prediction with someone else's target. Squaring gives 0,1,4,1,0,1,4,1,0; their sum is 12, and the mean over all nine entries is $12/9 = 1.333\dots$

NumPy followed its rule exactly. The mistake was that the two arrays had different orientations and nobody said which was intended.

The fix is the same idea as in Part 2: give both arrays the *same* shape before the subtraction. Either flatten the targets, or give the predictions a column orientation:

two correct versions

FIXED

```
((p - t.ravel()) ** 2).mean()      # both (3,)    → 0.0  
((p[:, None] - t) ** 2).mean()   # both (3, 1) → 0.0
```

Better still, make a mismatch impossible to miss. One line before the arithmetic turns the silent bug into a loud one:

the habit

PYTHON

```
assert p.shape == t.shape, (p.shape, t.shape)  
AssertionError: ((3,), (3, 1))
```



Why this one deserves ten minutes

Most mistakes in numerical code either crash or produce something obviously absurd. This one produces a number of a plausible size. In a training loop, the loss built this way is still a number, still decreases somewhat, and never reaches what it should — which looks like a tuning problem rather than a shape problem. It is one of the most common silent bugs in NumPy code (not the only one: integer division and accidental in-place changes are others we will meet). The defence is cheap: whenever two arrays meet in an element-wise operation, know both shapes, and `assert` them when it matters.

Three plots: line, scatter, and both together

Matplotlib is large. This course needs a small part of it, and that part fits on one page.

4

From session 9 onward many of our questions are about how one quantity changes with another: loss against training step, error against a parameter, model against data. Looking at those relationships is faster and more reliable than reading columns of numbers.

the two imports, and the smallest complete plot

PYTHON

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(-3, 3, 200)    # 200 evenly spaced values from -3 to 3, ends included
y = x ** 2                    # element-wise: y has shape (200,) too

plt.plot(x, y)                # joins the points (x[i], y[i]) in order
plt.show()
```

`plt.plot(x, y)` needs two arrays of the *same length* and draws straight segments between consecutive points. Two consequences: 200 points close together look like a smooth curve; and the points are joined in the order they appear, so `x` should be sorted — `linspace` guarantees that.

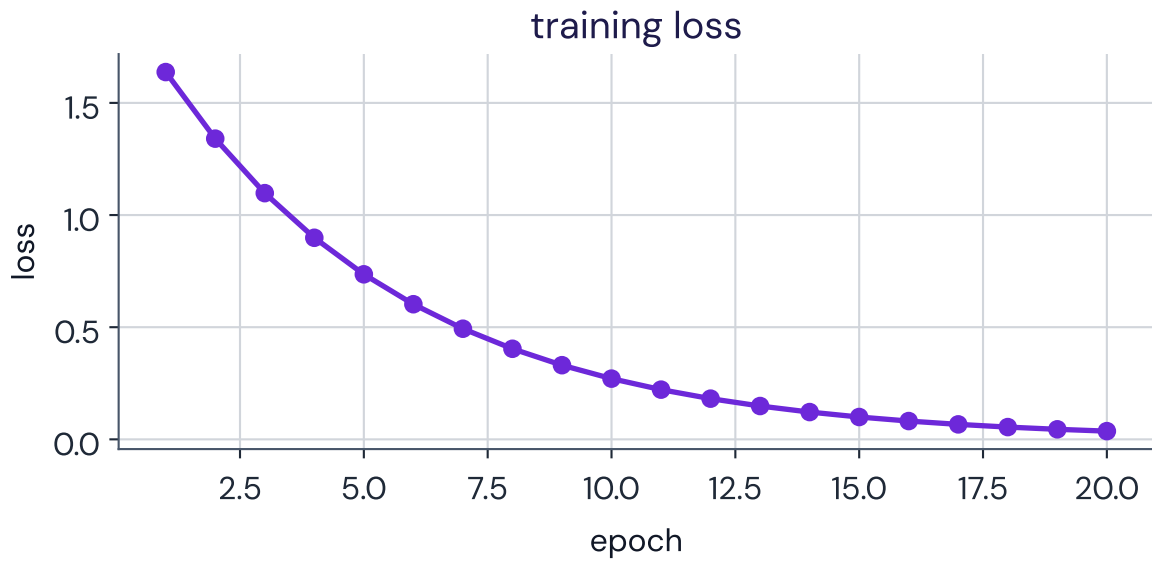
Plot one — a line, for a quantity that changes

a typical training-loss curve (made-up numbers, realistic shape)

PYTHON

```
epochs = np.arange(1, 21)
loss    = 2.0 * np.exp(-0.2 * epochs)

plt.plot(epochs, loss, marker="o")
plt.xlabel("epoch")
plt.ylabel("loss")
plt.title("training loss")
plt.grid(True)
plt.show()
```



What that code draws.



Label both axes, every time

A falling loss curve and a falling accuracy curve look identical, and mean opposite things. Only the axis label tells the reader which they are looking at. Treat `xlabel` and `ylabel` as part of the plot, not decoration.

Plot two — a scatter, for data points

synthetic data: 30 students, hours studied vs score

PYTHON

```
rng = np.random.default_rng(0)          # seed 0: everyone gets the same points
hours = rng.uniform(0, 10, 30)           # (30,)
scores = 8 * hours + rng.normal(0, 6, 30) # (30,): a trend of 8 points/hour, plus
noise

plt.scatter(hours, scores)
plt.xlabel("hours studied")
plt.ylabel("exam score")
plt.show()
```

`scatter` draws each point separately and joins nothing, so the order of the data does not matter. Use it for observations; use `plot` for a function evaluated on a sorted grid.



We know the answer here — on purpose

We *generated* this data with a slope of 8, so we know the true relationship. Real data never comes with that label. Having it here lets us check whether a method recovers the truth, which is exactly what we do in Part 5.

Plot three — two things on one pair of axes

Any number of `plot` and `scatter` calls before one `show()` draw on the same axes. Give each a `label=` and call `legend()` so the reader knows which is which.

data against one guessed line

PYTHON

```
xs = np.linspace(0, 10, 200)                # a sorted grid for the line

plt.scatter(hours, scores, label="data")
plt.plot(xs, 6 * xs, color="red", label="guess: w = 6")
plt.xlabel("hours studied")
plt.ylabel("exam score")
plt.legend()
plt.show()
```

Note that the line is drawn over `xs`, not over `hours`. The `hours` array is in random order; plotting a line through it would join points in that random order. For a straight line you would not notice, but for any curved model you would get a scribble.

PART 5 · 8 MINUTES

Broadcasting on purpose: many models at once

The every-with-every table from Case C, used deliberately.

5

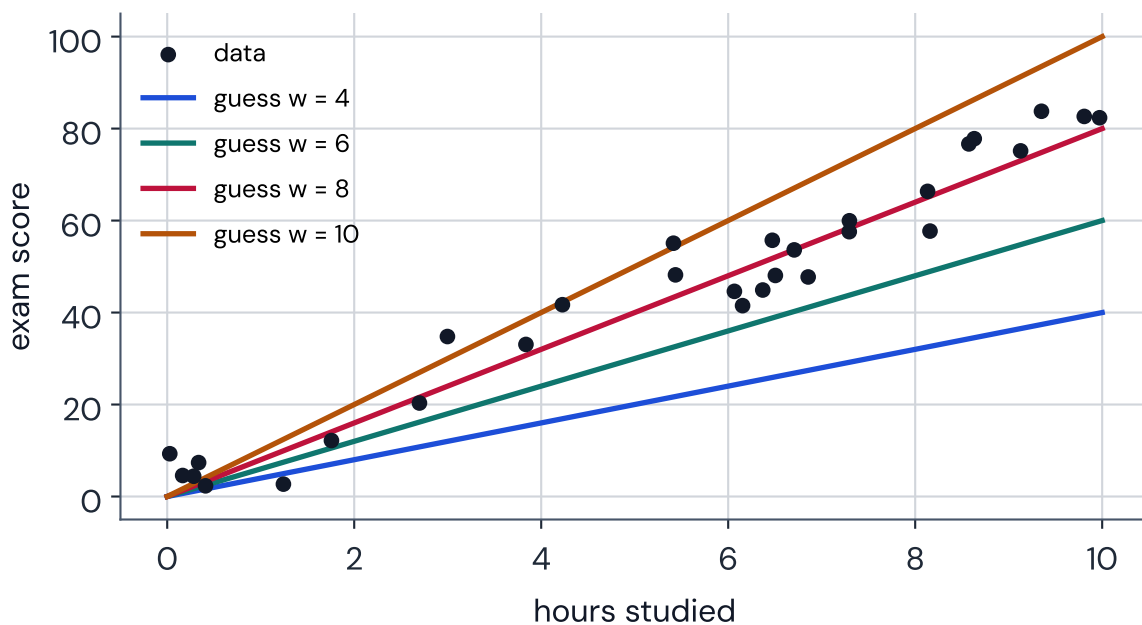
Is 6 a good slope? Rather than test guesses one at a time in a loop, put the candidate slopes in a **column** and let broadcasting combine them with the **row** of x-values:

four candidate lines in one expression

PYTHON

```
w = np.array([4., 6., 8., 10.])[:, None]  # (4, 1) - one candidate per row
lines = w * xs                           # (4, 1) * (200,) → (4, 200)

plt.scatter(hours, scores, color="black", label="data")
for k in range(4):
    plt.plot(xs, lines[k], label=f"guess w = {w[k, 0]:g}")
plt.xlabel("hours studied")
plt.ylabel("exam score")
plt.legend()
plt.show()
```



Row k of `lines` is the line for candidate k . The eye already says $w = 8$ fits best.

Now replace the eye with a number. The same pattern, applied to the data instead of the grid, gives every candidate's error on every student:

error of every candidate on every data point

PYTHON

```
errors = w * hours - scores  # (4, 1) * (30,) - (30,) → (4, 30)
mse = (errors ** 2).mean(axis=1)  # average along each row → (4,)
print(mse.round(1))
[721.4 223.5  34.3 153.8]
```

Row k of `errors` holds candidate k 's thirty errors; `mean(axis=1)` averages *across* each row, leaving one number per candidate. The slope 8 has by far the smallest error. Compare with Part 3: it is the same `(k, 1)` -with- `(n,)` mechanism. There it was an accident; here a column of candidates against a row of data is exactly what we want.

From four candidates to 281

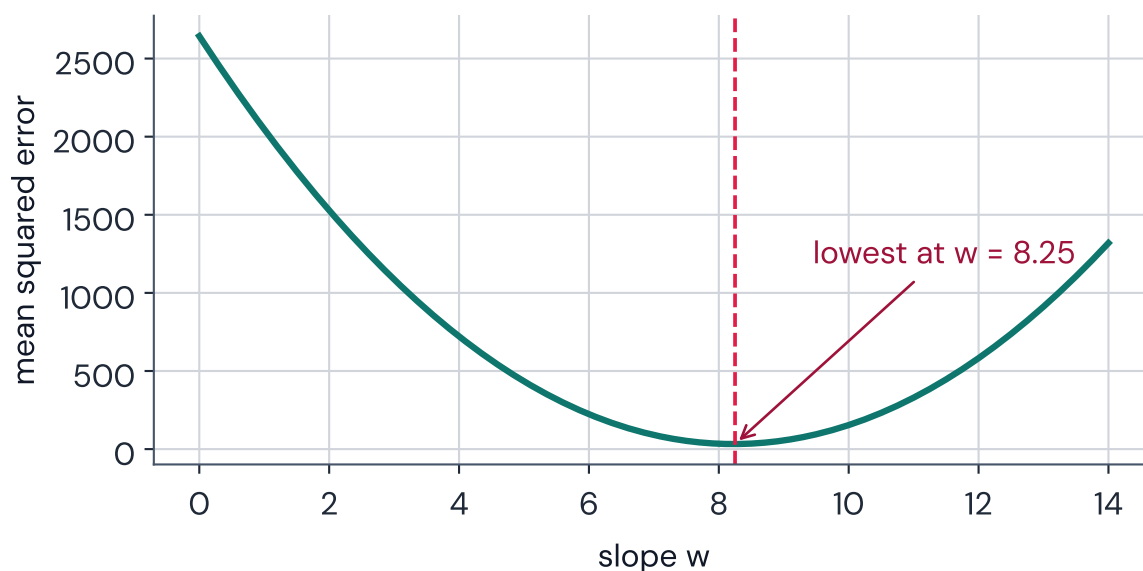
sweep the slope finely, plot the error, find the lowest point

PYTHON

```
ws = np.linspace(0, 14, 281) # steps of 0.05
mse = ((ws[:, None] * hours - scores) ** 2).mean(axis=1) # (281, 30) → (281,)

plt.plot(ws, mse)
plt.xlabel("slope w")
plt.ylabel("mean squared error")
plt.show()

print(ws[mse.argmin()])
8.25
```



Error as a function of the slope: a single valley. This is a *loss curve* over one parameter.

`argmin` returns the *position* of the smallest value; indexing `ws` with it gives the slope. The answer, 8.25, is close to the true 8 but not equal to it — the noise in thirty points shifts the best fit slightly, and the grid only checks multiples of 0.05. (On Lab 4 you can confirm that the exact best slope for this data is about 8.23.)

WHAT THIS COSTS, AND WHY SESSION 9 EXISTS

We found the best slope by trying 281 of them. That is feasible for one parameter. With two parameters on the same grid it is $281^2 \approx 79,000$ trials; with thirteen it is more than 10^{31} . Trying everything stops working almost immediately.

Look at the curve instead: at any point it slopes toward the minimum. Session 9 turns that observation into a method — follow the slope downhill — which needs no grid at all.

LAB 4 • 16 MINUTES IN CLASS

Vectorise five loops, time both, and draw one curve

6

Lab 4 is a separate handout (notebook, HTML and PDF). Five jobs written twice, once as a loop and once as an array operation, timed on your own machine so the speed-up is a number you measured. Then one labelled plot.



Same rules as before

Finish in the room; put your hand up after three minutes stuck. Nothing in the lab is a prerequisite for Friday.

Before Friday

1. **Finish Lab 4** and write your timing ratio at the top. Bring that number.
2. **Orientation drill.** For each of `(3,)`, `(3, 1)` and `(1, 3)`, write on paper the result shape of combining it with each of the others. Nine cases; check them in Python. If any surprised you, go back to the cards in Part 2 and find the one that explains it.
3. **Break it on purpose.** In the Part 5 sweep, remove the `[:, None]`. Read the error, explain it with the two-step rule, and say why reshaping `hours` instead would also work.



Session 8 — Friday 02 October, 11:30

The layer is a matrix multiply. $X @ W + b$ — the $+ b$ is Case A from today — is one complete layer of a neural network. You will compute a small layer by hand, then in one line of NumPy, and check that the two agree.